Planetaire Mono

B612 LETTERFORMS
HACK INFRASTRUCTURE
NERD FONT ICONS

A beautiful, highly legible monospace font
for terminals, editors, and agentic work

Version 0.1.2 · 2026-06-08
github.com/jlevy/planetaire

FEATURES

B612 base for letterforms
(extended Latin, Greek, Cyrillic)
Punctuation and symbols from Hack
12,000+ icons from Nerd Fonts
Modified zero (0) for legibility

WEIGHTS

Regular (400)
Italic (400)
Medium (500)
Medium Italic (500)
SemiBold (600)
SemiBold Italic (600)
Bold (700)
Bold Italic (700)
ExtraBold (800)
ExtraBold Italic (800)

CREDITS

Planetaire Mono packaged and maintained by **Joshua Levy**
B612 Mono letterforms by **Intactile Design** for **Airbus**
Hack base punctuation, symbols, and metrics by **Chris Simpkins**
Nerd Fonts 12,000+ icons by **Ryan McIntyre**
Dotted zero inspired by the **B612** fork by **Carlos Eduardo de Paula**

ABOUT B612

B612 began not as a typeface but as an aviation research program. In 2010 Airbus, ENAC (the French civil aviation university), and the Université de Toulouse III set out to define and validate an “aeronautical font” for cockpit screens: text a pilot can read correctly while fatigued, at oblique angles, or under vibration, glare, or near-darkness.

The shapes were derived experimentally before they were drawn. Jean-Luc Vinot (ENAC) and Sylvie Athènes (Toulouse III) built confusion matrices of when and how characters get misread (“Legible, are you sure?” at CHI 2012). In their controlled study, the prototype that became B612 drew slightly more correct reads than Verdana and clearly outperformed the legacy avionics font. Airbus then commissioned the Montpellier interface studio Intactile Design (Nicolas Chauveau, Thomas Paillot, and Jonathan Favre-Lamarine) to draw the full family of eight variants.

B612 is named for the asteroid home of the Little Prince, a nod to Saint-Exupéry, himself an aviator.

B612´s unusual character is a humanist answer to an instrument-panel problem. Where earlier cockpit fonts went monolinear and rigid, B612 keeps stroke contrast, opens counters, and lengthens ascenders and descenders. Each word’s silhouette resolves quickly. At stroke junctions it carries small notches (light traps) that keep joins from filling in on bright, low-contrast displays. The result has quietly human character quirks that are grounded in measured legibility gains rather than style. In 2017, B612 was released as open source through the Eclipse Polarsys project.

However, B612 alone is not a fully usable document or application font. The versions in circulation, including the one on Google Fonts, have uneven symbol coverage that is awkward for terminal and programming use. And the published version has an undotted zero that is easy to confuse with a capital O.

ABOUT PLANETAIRE MONO

Planetaire Mono arises from this need. It merges B612´s letters and digits into Hack Nerd Font’s base. It also adds more weights and a dotted zero:

- **B612 letterforms** for letters, digits, and extended Latin, Greek, and Cyrillic.
- **Hack punctuation and symbols** for {}[]()<> and the rest.
- **Ten variants across five weights** (400/500/600/700/800), including added SemiBold (600) and ExtraBold (800) weights, the latter for terminal bold (see Weights).
- **12,000+ Nerd Font icons** (Powerline, Font Awesome, Devicons) in the Extended package.
- **A dotted zero:** B612´s zero with a center dot for clear 0 vs O, in circle (default) and rectangle (ss01) variants.

With these changes it has become one of the most beautiful and genuinely functional monospace fonts I’ve seen. I’ve used dozens of terminal fonts over the years, and Planetaire Mono is now what I use every day.

Planetaire Typeface Specification

DESIGN

Classification	Monospace (fixed)
Letterforms	B612 Mono (humanist)
Packages	Extended, Text
Styles	10 (5 weights × 2)
Units per em	2000
Advance width	1204 (0.602 em)
Italic angle	0°, true italics

WEIGHTS

Regular	400
Medium	500
SemiBold	600
Bold	700
ExtraBold	800

VERTICAL METRICS (per 2000 em)

Cap height	1458
x-height	1094
Ascender	1856
Descender	-472
Line gap	0

GLYPH COVERAGE

Extended	12,138 glyphs
Text	1,317 glyphs

FORMATS

Local install	TTF
Web	WOFF2 + @font-face CSS

OPENTYPE

default	circle-dot zero
ss01 / zero	rectangle-dot zero

LICENSE

All packages	SIL OFL 1.1

Planetaire Text Sample

Planetaire Mono at various sizes, showing B612’s distinctive letterforms optimized for readability at small sizes and on low-resolution displays.

The quick brown fox jumps over the lazy dog. Pack my box with five dozen liquor jugs. How vexingly quick daft zebras jump!

In the great void between stars, instruments must be read without error. Every glyph must be unambiguous: the digit 0 distinct from the letter O, the numeral 1 clearly not a lowercase l or uppercase I. B612 was born from this requirement. Originally designed for cockpit displays at Airbus, where a misread character could mean the difference between FL350 and FL850. Planetaire Mono inherits that precision and pairs it with the full symbol coverage a programmer needs: braces, brackets, pipes, arrows, and 12,000 icons ready for a modern terminal.

FRENCH · GERMAN · SPANISH · TURKISH

Les naïfs ægithales hâtifs pondent au zéphyr joyeux. Falsches Üben von Xylophonmusik quält jeden größeren Zwerg. El veloz murciélago hindú comía feliz cardillo y kiwi. Pijamalı hasta yağız şoföre çabucak güvendi.

GREEK

Ξεσκεπάζω τὴν ψυχοφθόρα βδελυγμία.

CYRILLIC

Съешь же ещё этих мягких французских булок, да выпей чаю. Широкая электрификация южных губерний даст мощный толчок подъёму сельского хозяйства.

Planetaire Iconic Texts

ALAN TURING · “COMPUTING MACHINERY AND INTELLIGENCE” (1950)

I propose to consider the question, “Can machines think?” This should begin with definitions of the meaning of the terms “machine” and “think.” The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words “machine” and “think” are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, “Can machines think?” is to be sought in a statistical survey such as a Gallup poll. But this is absurd.

Network Working Group
Request for Comments: 1

Steve Crocker
UCLA
7 April 1969

Title: **Host Software**
Author: **Steve Crocker**
Installation: UCLA
Date: 7 April 1969
Network Working Group Request for Comment: 1

Introduction

The software for the ARPA Network exists partly in the IMPs and partly in the respective HOSTs. BB&N has specified the software of the IMPs and it is the responsibility of the HOST groups to agree on HOST software.

During the summer of 1968, representatives from the initial four sites met several times to discuss the HOST software and initial experiments on the network. There emerged from these meetings a working group of three, Steve Carr from Utah, Jeff Rulifson from SRI, and **Steve Crocker** of UCLA, who met during the fall and winter. The most recent meeting was in the last week of March in Utah. Also present was Bill Duvall of SRI who has recently started working with Jeff Rulifson.

Somewhat independently, Gerard DeLoche of UCLA has been working on the HOST-IMP interface.

I present here some of the tentative agreements reached and some of the open questions encountered. Very little of what is here is firm and reactions are expected.

I. A Summary of the IMP Software

Messages

Information is transmitted from HOST to HOST in bundles called messages. A message is any stream of not more than 8080 bits, together with its header. The header is 16 bits and contains the following information:

Destination	5 bits
Link	8 bits
Trace	1 bit
Spare	2 bits

The destination is the numerical code for the HOST to which the message should be sent. The trace bit signals the IMPs to record status information about the message and send the information back to the NMC (Network Measurement Center, i.e., UCLA). The spare bits are unused.

Planetaire Code Specimen: microGPT

Andrej Karpathy’s microGPT: a complete GPT training loop and inference engine in 200 lines of pure Python.

```python
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os       # os.path.exists
import math     # math.log, math.exp
import random   # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: list[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # scalar value of this node calculated during forward pass
        self.grad = 0                   # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children       # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad
```

```python
# Initialize the parameters, to store the knowledge of the model
n_layer = 1     # depth of the transformer neural network (number of layers)
n_embd = 16     # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4      # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(vocab_size,
n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params into a single list[Value]
print(f"num params: {len(params)}")

# Define the model architecture: a function mapping tokens and parameters to logits over what comes next
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm -> rmsnorm, no biases, GeLU -> ReLU
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)
```

```python
    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0

    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")
```

Planetaire Terminal

```python
import math

def analyze_trajectory(altitude: float, velocity: float) -> dict:
    """Calculate orbital parameters."""

    G = 6.674e-11  # gravitational constant
    M = 5.972e24

    if altitude > 400_000:
        orbit_type = "LEO"
    elif altitude > 35_786_000:
        orbit_type = "GEO"

    period = 2 * math.pi * math.sqrt(altitude**3 / (G * M))

    return {"type": orbit_type, "period": period, "v": velocity}
```


```python
import math

def analyze_trajectory(altitude: float, velocity: float) -> dict:
    """Calculate orbital parameters."""

    G = 6.674e-11  # gravitational constant
    M = 5.972e24

    if altitude > 400_000:
        orbit_type = "LEO"
    elif altitude > 35_786_000:
        orbit_type = "GEO"

    period = 2 * math.pi * math.sqrt(altitude**3 / (G * M))

    return {"type": orbit_type, "period": period, "v": velocity}
```


```
planetaire $ eza -l --icons=always . ./docs/specimen/
.:
drwxr-xr-x@     - levy 15 Feb 23:07  devtools
drwxr-xr-x@     - levy 15 Feb 23:07  docs
drwxr-xr-x@     - levy 15 Feb 23:07  fonts
.rw-r--r--@ 7.6k levy 16 Feb 09:14  LICENSE
.rw-r--r--@ 1.3k levy 16 Feb 09:14  Makefile
.rw-r--r--@ 6.2k levy 16 Feb 09:14  pyproject.toml
.rw-r--r--@ 6.4k levy 15 Feb 23:07  README.md
drwxr-xr-x@     - levy 15 Feb 23:07  src
.rw-r--r--@  63k levy 16 Feb 09:14  uv.lock
./docs/specimen/:
.rw-r--r--@ 188k levy 16 Feb 09:14  planetaire-mono-specimen.pdf
.rw-r--r--@ 9.1k levy 15 Feb 23:07  planetaire-mono-specimen.typ
```

```
planetaire $ python -c "print('Hello from Planetaire Mono!')"
Hello from Planetaire Mono!

planetaire $ git log --oneline -3
5bd69c5 Switch B612 source to original polarsys/b612
a1c8e3f Add font comparison and regression detection
e927d01 Refactor merge pipeline for original B612
```

```
planetaire $ eza -l --icons=always . ./docs/specimen/
.:
drwxr-xr-x@     - levy 15 Feb 23:07  devtools
drwxr-xr-x@     - levy 15 Feb 23:07  docs
drwxr-xr-x@     - levy 15 Feb 23:07  fonts
.rw-r--r--@  7.6k levy 16 Feb 09:14  LICENSE
.rw-r--r--@  1.3k levy 16 Feb 09:14  Makefile
.rw-r--r--@  6.2k levy 16 Feb 09:14  pyproject.toml
.rw-r--r--@  6.4k levy 15 Feb 23:07  README.md
drwxr-xr-x@     - levy 15 Feb 23:07  src
.rw-r--r--@   63k levy 16 Feb 09:14  uv.lock
./docs/specimen/:
.rw-r--r--@  188k levy 16 Feb 09:14  planetaire-mono-specimen.pdf
.rw-r--r--@  9.1k levy 15 Feb 23:07  planetaire-mono-specimen.typ

planetaire $ python -c "print('Hello from Planetaire Mono!')"
Hello from Planetaire Mono!

planetaire $ git log --oneline -3
5bd69c5 Switch B612 source to original polarsys/b612
a1c8e3f Add font comparison and regression detection
e927d01 Refactor merge pipeline for original B612
```

Planetaire Character Set

BASIC LATIN UPPERCASE: from B612

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

BASIC LATIN LOWERCASE: from B612

a b c d e f g h i j k l m n o p q r s t u v w x y z

DIGITS: 0-9 from B612 (zero modified with center dot for O/0 disambiguation)

0 1 2 3 4 5 6 7 8 9

PUNCTUATION AND SYMBOLS: from Hack

! “ # $ % & ‘ ( ) * + , - . / : ; < = > ? @ [\ ] ^ _
` { | } ~

EXTENDED LATIN: from B612

À Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß

à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ø ù ú û ü ý þ ÿ

Ā ā Ă ă Ą ą Ć ć Ĉ ĉ Ċ ċ Č č Ď ď Đ đ Ē ē Ĕ ĕ Ė ė Ę ę Ě ě

Ğ ğ Ġ ġ Ģ ģ Ĥ ĥ Ħ ħ Ĩ ĩ Ī ī Ĭ ĭ Į į İ ı Ĳ ĳ Ĵ ĵ Ķ ķ

GREEK: from B612

Α Β Γ Δ Ε Ζ Η Θ Ι Κ Λ Μ Ν Ξ Ο Π Ρ Σ Τ Υ Φ Χ Ψ Ω

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

CYRILLIC: from B612

А Б В Г Д Е Ж З И Й К Л М Н О П Р С Т У Ф Х Ц Ч Ш Щ Ъ Ы Ь Э Ю Я

а б в г д е ж з и й к л м н о п р с т у ф х ц ч ш щ ъ ы ь э ю я

Planetaire Weight Comparison

REGULAR (400)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

ITALIC (400)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

MEDIUM (500)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

MEDIUM ITALIC (500)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

SEMIBOLD (600)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

SEMIBOLD ITALIC (600)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

BOLD (700)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

BOLD ITALIC (700)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

EXTRABOLD (800)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

EXTRABOLD ITALIC (800)

The quick brown fox jumps over the lazy dog

0123456789 AaBbCcDd {[(>)]} !@#$%

Planetaire Size Waterfall

Planetaire Mono from caption to display size, holding its proportions throughout.

8 Planetaire Mono

9 Planetaire Mono

10 Planetaire Mono

11 Planetaire Mono

12 Planetaire Mono

14 Planetaire Mono

16 Planetaire Mono

20 Planetaire Mono

24 Planetaire Mono

30 Planetaire Mono

36 Planetaire Mono

44 Planetaire Mono

LEGIBILITY AT DISPLAY SIZE

Il1| O0o 0123

Planetaire Legibility

CHARACTER DISAMBIGUATION

I l 1 |	uppercase I · lowercase l · digit 1 · pipe
O 0 o	uppercase O · digit 0 · lowercase o
r n m	r · n · m — clearly distinct in B612
5 S 8 B	digit 5 vs S · digit 8 vs B
2 Z 6 G	digit 2 vs Z · digit 6 vs G
; :	semicolon vs colon — distinct dot size and spacing
( ) { } [ ]	parens vs braces vs brackets
- – —	hyphen-minus vs en dash vs em dash
“ ” "	left double quote vs right double quote vs straight double quote
‘ ’ ' `	left single quote vs right single quote/ apostrophe vs straight quote vs backtick

ZERO DOT VARIANTS (OpenType)

Default (circle dot)

0 0 O o

FL350 FL850 10.0.0.1

ss01 / zero (rectangle dot)

0 0 O o

FL350 FL850 10.0.0.1

BRACKET AND DELIMITER PAIRS

( ) { } [ ] < > « » ‹ ›

MATHEMATICAL AND PROGRAMMING OPERATORS

+ - * / = ≠ ≤ ≥ ± × ÷ → ← ↑ ↓

Planetaire Symbols and Special Characters

STANDARD PUNCTUATION

. , ; : ! ? - - – ( ) [ ] { } / \ @ # $ % ^ & *
_ + = ~ `

QUOTES AND APOSTROPHES

" ' ` “ ” ‘ ’ « » ‹ ›

straight double · straight single · backtick · left double · right double · left single · right single/apostrophe · guillemets · single guillemets

TYPOGRAPHIC AND SPECIAL CHARACTERS

§ ¶ © ® ™ ° · … † ‡ ¤ ¢ £ ¥ € ¬ ¦

section · pilcrow · copyright · registered · trademark · degree · middle dot · ellipsis · dagger · double dagger · currency · cent · pound · yen · euro · not · broken bar

SLASHES AND STROKES

/ \ | ¦ ⁄ ∕

solidus · reverse solidus · vertical bar · broken bar · fraction slash · division slash

BOX DRAWING AND BLOCK ELEMENTS

─ │ ┌ ┐ └ ┘ ├ ┤ ┬ ┴ ┼ ═ ║ ╔ ╗ ╚ ╝ ╠ ╣ ╦ ╩ ╬ ▀ ▄ ▌ ▐ ▐
░ ▒ ▓

Nerd Font Icons

A selection of the 12,000+ Nerd Font icons included via Hack Nerd Font. Each icon shown with its Unicode codepoint.

UI AND COMMON

F002 F005 F007 F008 F009 F013 F015 F017 F019 F021 F023 F024 F025 F026

F028 F02B F02D F030 F03D F040 F05B F06C F071 F073 F074 F07B F07C F080

F085 F0C5 F0C7 F0C9 F0D0 F0E7 F0EB F0F3 F0FE F11C F120 F121 F126 F130

F15B F15C F187 F188 F1B2 F1B3 F1C0 F1D3 F1DE F1E0 F1EB F233 F27A F296

FILE TYPES

E60A E60E E612 E621 E622 E623 E624 E635 E63A E640 E648 E64A E64B E656

E657 E65E E661 E667 E668 E6A8 E697 E69B E6A0 E6B2 E6A1 E6B4 E6A7 E6B7

DEVELOPMENT

E702 E718 E73C E74E E781 E791 E7A8 E749 E781 E606 E615 E796 E60B E617

E73E E753 E70C E749 E755 E706 E628 E614 E70E E711 E718 E62A E629 E711

WEATHER

E30D E312 E318 E31A E320 E335 E33D E342 E343 E344 E366 E36E E370 E3A9

POWERLINE

E0A0 E0A1 E0A2 E0B0 E0B1 E0B2 E0B3 E0B4 E0B6 E0B8 E0BA E0BC E0BE E0C0

E0C2 E0C4 E0C6 E0C8 E0CA E0CC E0CE E0D0 E0D2 E0D4

Planetaire Provenance and License

Source Fonts

B612 Mono	Designed by Intactile Design for Airbus. Optimized for legibility in cockpit displays. Planetaire Mono takes its letters (A-Z, a-z), digits 0-9, and extended Latin/Greek/Cyrillic glyphs from B612. The zero glyph receives a center dot in post-processing for O/0 disambiguation, with circle (default) and rectangle (ss01) variants.
Hack	Chris Simpkins' typeface designed for source code. Provides the base font structure: punctuation, symbols, metrics, and Nerd Font integration.
Nerd Fonts	Ryan McIntyre's icon patching project. 12,000+ developer icons including Powerline, Font Awesome, Devicons, Material Design, and more, all included via the Hack Nerd Font base.

Build Pipeline

Planetaire Mono is built with a custom Python pipeline using fontTools. For each weight variant, the pipeline loads Hack Nerd Font as the base, merges B612 letter and digit glyphs by Unicode range, adds a center dot to B612's zero for O/0 disambiguation, renames the result, and applies post-processing fixes. Medium and SemiBold weights are generated from Regular, and ExtraBold from Bold, via FontForge emboldening.

Two Packages: Extended and Text

Planetaire Mono ships in two packages built from the same letterforms:

- **Planetaire Mono Extended**: the full build with all ~12,000 Nerd Font icons and Powerline, for terminals and coding.
- **Planetaire Mono Text**: a lightweight web subset (letters, punctuation, box-drawing, block elements, geometric shapes) that drops the Private-Use icons. About **55 KB per weight** in WOFF2, roughly 18× smaller, and shipped with a ready `@font-face` stylesheet.

Planetaire Mono Text

```
The quick brown fox jumps over the lazy dog
ABCDEFGHIJKLMNOPQRSTUVWXYZ abcdefghijklmnopqrstuvwxyz
0123456789 !@#$%^&*()[]{} <>=+ — Il1| O0o
Greek ΑΒΓ αβγ · Cyrillic АБВ абв · Box ┌┬┐ │ ├┼┤ └┴┘ ▓▒▒
```

License

Planetaire Mono is released under the **SIL Open Font License 1.1 (OFL-1.1).**

The OFL allows free use, modification, and redistribution of the font, including in commercial products, provided that modified versions are not sold by themselves and carry a different name.

The constituent fonts carry the following licenses:
- **B612 Mono**: SIL Open Font License 1.1 and Eclipse Public License 2.0
- **Hack**: MIT License
- **Nerd Fonts** patches: MIT License

Spacing Review

Planetaire Mono is built to a single cell width: every glyph (and intentional double-width glyphs at exactly 2x) shares one advance. B612's letters and the FontForge-emboldened weights are normalized to that cell, recentered, and condensed only where ink would otherwise bleed. The two panels below are the visual proof. Review them to confirm no glyph is trimmed and all weights align.

STANDARD CODING CHARACTERS: TRUE CELL WIDTHS

Red rules mark each glyph’s advance. Equal-width cells with ink inside mean a clean monospace grid, nothing trimmed.

A B C D E F G H I J K L M N O P Q R S T U
V W X Y Z

a b c d e f g h i j k l m n o p q r s t u
v w x y z

0 1 2 3 4 5 6 7 8 9

! " # $ % & ' ( ) * + , - . / : ; < = > ? @

[\ ] ^ _ ` { | } ~

Cell-filling glyphs (box-drawing, powerline) reach the rules by design:

─ │ ┌ ┐ └ ┘ ├ ┤ ┬ ┴ ┼ █ ▓ ▒ ░ ▶ ◀

STANDARD CODING CHARACTERS: TRUE CELL WIDTHS (ITALIC)

Red rules mark each glyph’s advance. Equal-width cells with ink inside mean a clean monospace grid, nothing trimmed.

A B C D E F G H I J K L M N O P Q R S T U
V W X Y Z

a b c d e f g h i j k l m n o p q r s t u
v w x y z

0 1 2 3 4 5 6 7 8 9

! " # $ % & ' ( ) * + , - . / : ; < = > ? @

[\ ] ^ _ ` { ¦ } ~

Cell-filling glyphs (box-drawing, powerline) reach the rules by design:

─ │ ┌┐ └┘ ├┤ ┬┴┼ █ ▒ ▓ ▶ ◀

WEIGHT ALIGNMENT: EVERY WEIGHT IS THE SAME WIDTH

The same string in all ten variants; the red rule marks each line's right edge. A single vertical line means identical width across every weight.

REGULAR (400)	`if (x == 0) { i = l1|O; }`
ITALIC (400)	*`if (x == 0) { i = l1¦O; }`*
MEDIUM (500)	`if (x == 0) { i = l1|O; }`
MEDIUM ITALIC (500)	*`if (x == 0) { i = l1¦O; }`*
SEMIBOLD (600)	`if (x == 0) { i = l1|O; }`
SEMIBOLD ITALIC (600)	*`if (x == 0) { i = l1¦O; }`*
BOLD (700)	`if (x == 0) { i = l1|O; }`
BOLD ITALIC (700)	*`if (x == 0) { i = l1¦O; }`*
EXTRABOLD (800)	`if (x == 0) { i = l1|O; }`
EXTRABOLD ITALIC (800)	*`if (x == 0) { i = l1¦O; }`*